

데이터 분석 파이프라인 설계

CLIP 01

파이프라인
구조 & 세팅

CLIP 02

데이터 수집
탐색 에이전트

CLIP 03

분석 방향
설계 에이전트

CLIP 04

대시보드·보고서
에이전트 완성

P02에서 익힌 것

- CLAUDE.md**

프로젝트 규칙과 실행 지침을 담는 파일
- Skills**

반복 절차를 에이전트가 꺼내 쓰는 도구 모음
- Subagents**

역할별로 분리해서 실행하는 에이전트



이번 챕터 — P03 CH01

세 가지를 하나의 분석 흐름으로

이번 장에서는 이 세 가지를 하나의 분석 흐름으로 묶어 봅니다

이 7단계 구조에 대해

교육용 참조 구조

**이 7단계는 분석 흐름을
끊어서 이해하기 위한 분해입니다**

복잡한 과정을 한 번에 보지 않고, 단계별로 끊어서 이해하기 위해 나눴습니다. 7이라는 숫자 자체가 중요한 것이 아닙니다.

실무에서는 단순해질 수 있습니다

**단계를 합치거나 생략해도
됩니다**

데이터 규모, 팀 구조, 프로젝트 성격에 따라 2~3단계로 압축하거나 특정 단계를 건너뛸 수 있습니다.

흐름이 중요합니다

**단계 개수보다 분석 방향이
먼저 있어야 합니다**

문제 정의 → 지표 설계 → 검증 → 전달. 이 흐름이 유지되는 한 구조는 조정할 수 있습니다.

이 챕터의 학습 목표

**에이전트로 분석 흐름을
단계별로 실행해보기**

7개로 나눈 이유는 각 단계의 역할과 산출물을 명확히 보기 위해서입니다. 이 구조를 직접 써본 뒤 필요에 맞게 조정하면 됩니다.

이번 챕터에서 만드는 것



7단계 흐름을 직접 만들며 각 에이전트의 역할과 산출물 연결 방식을 확인합니다

CLIP 01

파이프라인 구조 & 세팅

7단계가 왜 필요한지, 어떤 폴더 구조로 시작하는지
그리고 CLAUDE.md를 어떻게 쓰는지 다룬다.

복잡한 분석일수록, 단일 Claude의 한계

역할 분리 + 단계 분리 — 파이프라인

단순 대화 방식

사용자

CSV 분석부터 대시보드·보고서 작성까지 한 번에 요청

- ✗ Claude 한 명이 처음부터 끝까지
- ✗ 컨텍스트에 모든 게 쌓여 중반부터 느려짐
- ✗ 중간에 실패하면 처음부터 다시
- ✗ 다음번에 또 설명해야 함

파이프라인 방식

사용자

/run-pipeline

- ✓ 7개로 역할을 나눠 각 단계를 분리
- ✓ 각 에이전트는 자기 단계만 처리
- ✓ 실패한 단계만 재실행
- ✓ 팀 전체가 같은 방식으로 반복 사용

이 강의에서 쓰는 참조 구조 — 7단계로 분석 흐름을 나눈 예시

각 단계 결과물이 다음 단계의 입력이 됨

01	@data-ingestion	데이터 파일 읽고 컬럼 구조 파악 — 결측치·이상치·중복 진단 후 최소한의 전처리만	01_dataset_profile.md
02	@eda	컬럼별 분포 확인 후 그룹·상관 탐색 — 주요 발견 3가지 + 분석 질문 도출	02_eda_report.md
03	@problem	패턴을 비즈니스 문제로 번역 — MECE 원인 계층 구조 + 데이터로 답할 분석 질문 Top 3	03_problem_definition.md
04	@metrics	분석 질문을 측정 가능한 지표로 확정 — 이름·계산식·단위·제외조건까지 정의	04_kpi_summary.md
05	@analysis	KPI를 실제 데이터로 검증 — 전체 현황에서 원인까지 파고드는 흐름으로 인사이트 도출	05_analysis_report.md
06	@dashboard	인사이트를 3초 안에 읽히는 인터랙티브 HTML 대시보드로 — 템플릿 기반으로 제작	06_dashboard.html
07	@report	분석 전체를 임원이 5분 안에 읽을 임원용 요약 보고서로 압축 — 액션 플랜 포함	07_executive_report.md

이 구조의 목적은 예쁘게 나누는 게 아니라, **중간 단계만 다시 실행하고 결과를 검증할 수 있게 만드는 것**입니다. EDA가 실패해도 data-ingestion은 재실행 안 해도 됩니다.

실무에서는 단계를 합치거나 생략할 수 있습니다. 이 구조는 분석 흐름을 끊어서 이해하기 위한 교육용 분해입니다.

파일 하나 = 에이전트 하나

YAML 프론트매터 + 마크다운 본문으로 구성

에이전트 = 파일 하나

```
.claude/agents/
|
├─ eda.md           ← @eda
├─ problem.md       ← @problem
├─ metrics.md        ← @metrics
└─ dashboard.md     ← @dashboard
```

파일 이름 = 호출 이름

파일 안의 구조

--- (YAML 프론트매터)

name · description · tools

model · permissionMode 선언

--- 아래 (마크다운 본문)

에이전트가 따를 작업 지침을 자연어로 작성

코드 한 줄 없이 자연어 선언만으로 전문 에이전트 완성

ai-pipeline-kit 폴더 구조

세 가지 폴더 + CLAUDE.md 하나로 구성

폴더 트리

```
ai-pipeline-kit/
├── CLAUDE.md
├── .claude/
│   └── agents/
│       ├── data-ingestion.md
│       ├── eda.md
│       ├── problem.md
│       ├── metrics.md
│       ├── analysis.md
│       ├── dashboard.md
│       └── report.md
│   └── skills/
│       ├── run-pipeline/SKILL.md
│       ├── dashboard-design/SKILL.md
│       └── visualize/SKILL.md
└── runs/
    └── latest/
        ├── input/  ← CSV 여기
        └── outputs/ ← 산출물
```

CLAUDE.MD

실행 순서와 규칙 선언

파이프라인 흐름과 실행 규칙을 자연어로 선언하는 파일

.CLAUDE/AGENTS/

7개 전문 에이전트

각 에이전트의 역할·모델·도구·산출물을 정의한 .md 파일

RUNS/LATEST/INPUT/

데이터 투입구

분석할 CSV·Excel·JSON 등을 이 폴더에 넣으면 시작

latest 이름은 고정 아님 — 날짜별 관리 시 runs/2025-05-20/처럼 교체 가능

RUNS/LATEST/OUTPUTS/

단계별 산출물

01_~07_ 번호 순으로 각 에이전트의 결과물이 쌓이는 곳



@data-ingestion

Step 01 · Data Profiling

역할 · 작업 순서

역할: '이 데이터로 무엇을 할 수 있는가'를 팀 전체가 같은 이해로 시작하게 만든다. 분석하지 않고 구조를 파악한다.

- **1단계:** runs/latest/input/ 에서 CSV 읽기 — 행/컬럼 수, 각 컬럼 이름·타입·역할 분류
- **2단계:** 데이터 품질 진단 — 결측치 심각도 3단계(5%/20% 기준), 이상치 IQR·3σ 기준, 중복 행, 날짜·수치 형식 이슈
- **3단계:** 전처리 필요성 판단 — 필요한 것만 수행하고 이유 명시, 처리 전후 5행 비교 출력
- **4단계:** 01_dataset_profile.md 저장

입력: runs/latest/input/*.csv

원칙 · 입출력

출력: 01_dataset_profile.md

- 분석하지 않는다 — 구조와 품질 파악이 전부
- 있는 그대로 보고한다 — 문제를 축소하지 않는다
- 불필요한 전처리는 하지 않는다 — 필요한 것만 처리하고 이유를 밝힌다
- 도메인 가정을 강제하지 않는다 — 데이터가 무엇인지는 데이터가 말하게 한다

model: claude-haiku-4-5 · **tools:** Read, Write, Bash, Glob · **permissionMode:** auto

이번 챕터에서 만드는 것



@data-ingestion → @eda 순서대로 실행

CLIP 02

데이터 수집 탐색 에이전트

@data-ingestion · @eda 두 에이전트를 만든다.

파일 하나 = 에이전트 하나.

들어오는 것

runs/latest/input/

CSV 파일

컬럼 의미·타입·결측 상태가
아직 확인되지 않은 상태



이번 CLIP에서 하는 일

@data-ingestion
컬럼 구조 파악 · 결측치 · 이상치 진단

@eda
분포 · 상관 · 주요 패턴 탐색



만들어지는 것

01_dataset_profile.md
데이터 구조 리포트

02_eda_report.md
패턴 탐색 리포트

이 두 파일이 있어야
Clip 03 시작 가능

Step 01 — @data-ingestion

데이터 읽기 + 컬럼 구조 파악 + 품질 진단

.CLAUDE/AGENTS/DATA-INGESTION.MD

```
----

name: data-ingestion
description: 데이터 파일을 받아 구조를 파악하고
            데이터 품질을 진단한 뒤, 다음 분석
            단계가 쓸 수 있는 프로파일을 만드는
            에이전트. 파이프라인의 첫 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-haiku-4-5
permissionMode: auto
----

# @data-ingestion

## 작업 순서
1. runs/latest/input/ 파일 읽기
2. 결측치 · 이상치 · 중복 진단
3. 필요한 전처리만 수행
4. 01_dataset_profile.md 저장
```

name

호출 이름 — @data-ingestion으로 호출

description

Claude가 이 에이전트를 언제 쓸지 판단하는 기준 — 구체적일수록 정확하게 작동

model: claude-haiku-4-5

구조 파악·진단 작업은 haiku로 충분 — 비용 절약

permissionMode: auto

파일 읽기·쓰기를 확인 없이 자동 처리

Step 02 — @eda

패턴 탐색 + 분석 질문 도출

.CLAUDE/AGENTS/EDA.MD

```
----
name: eda
description: 데이터 프로파일을 받아 탐색적 분석을
  수행하고 분석 방법론 후보와 핵심 질문을
  도출하는 에이전트. 두 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-haiku-4-5
permissionMode: auto
----

# @eda

## 인풋
- 01_dataset_profile.md 읽기
- runs/latest/input/ CSV 직접 읽기

## 작업
- 일변량 분석 (분포, 극단값)
- 다변량 분석 (상관, 그룹 비교)
- 핵심 발견 3개 + 분석 질문 도출

## 아웃풋
02_eda_report.md 저장
-----
```

일변량 분석
컬럼 하나씩 독립 분석 — 분포 형태, 평균·중앙값·극단값 확인

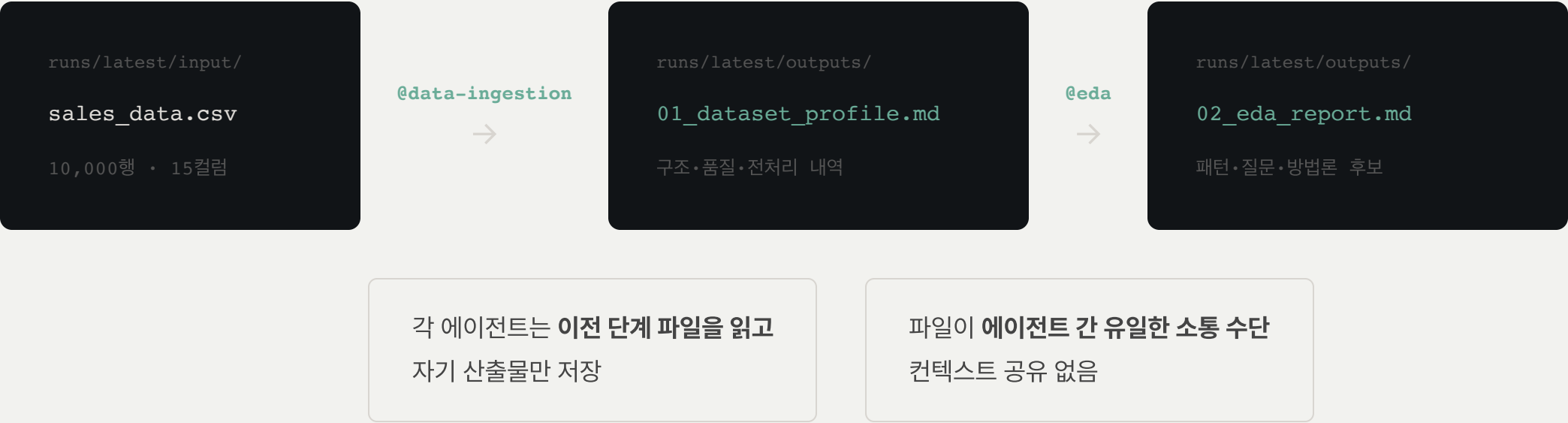
다변량 분석
컬럼 간 관계 탐색 — 상관관계, 그룹별 비교, 시계열 추세

인풋 → 아웃풋
01_dataset_profile.md
→ 02_eda_report.md

"숫자를 나열하지 않는다 — 패턴을 찾는다"

파일이 에이전트 간 유일한 소통 수단

앞 단계 산출물이 다음 단계 입력으로 연결



한 에이전트에는 한 역할만 맡깁니다

역할이 좁을수록 더 정확하게 작동하고, 실패해도 그 단계만 재실행

@data-ingestion	구조·품질 파악	분석 안 한다
@eda	패턴 탐색	결론 내리지 않는다
@problem	왜 일어났나 정의	무엇이 보였는지보다 왜 그런지
@metrics	지표 정의	모호함 없앤다
@analysis	데이터로 검증	지표 재정의 안 한다
@dashboard	인터랙티브 HTML 제작	3초 안에 읽히게
@report	임원 보고서 압축	새로운 발견 없음

실패 시 그 단계만 재실행

EDA가 실패해도 data-ingestion을 다시 안 해도 됨 — 산출물 파일이 체크
포인트 역할

역할 명확 → 환각 감소

@eda에 "차트 만들어"라고 안 씀 — 역할이 좁을수록 Claude가 정확하게
작동

파일로만 소통 → 컨텍스트 비대화 방지

에이전트 간 대화 없이 파일로만 소통 — 컨텍스트 창이 중간 과정으로 꽉 차
지 않음

CLIP 02를 마치면

raw CSV → 구조 파악 문서 + 탐색 리포트
@data-ingestion · @eda 두 에이전트가 순서대로 처리

이번 챕터에서 만드는 것



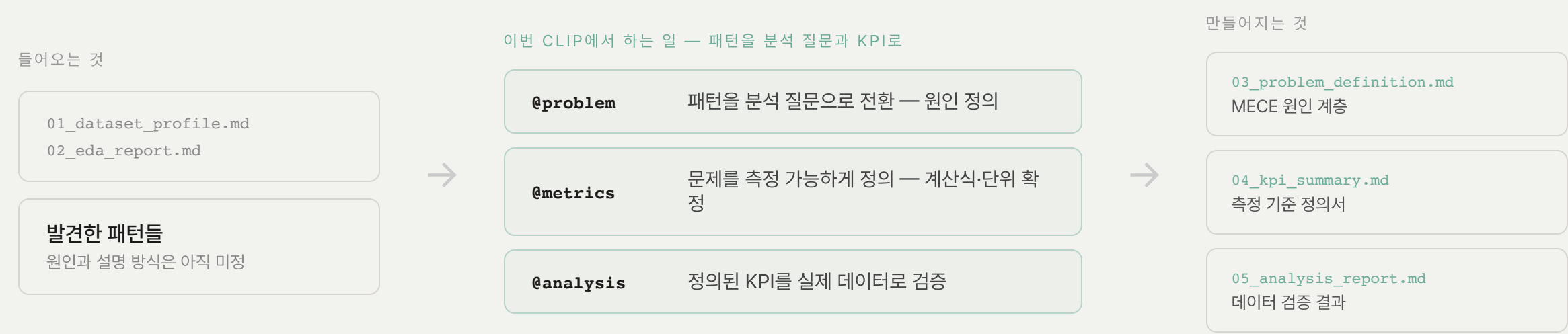
@problem · @metrics · @analysis 세 에이전트 실행

CLIP 03

분석 방향 설계 에이전트

문제 정의 → 지표 확정 → 데이터 검증.

@problem · @metrics · @analysis



Step 03 — @problem

EDA 패턴 → 비즈니스 문제로 번역 + MECE 원인 구조화

.CLAUDE/AGENTS/PROBLEM.MD

```
----
name: problem
description: EDA 결과를 받아 비즈니스 관점의
             문제를 정의하고, MECE 원인 계층
             구조와 해결 방향을 도출. 세 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-haiku-4-5
permissionMode: auto
----

## 인풋
- 02_eda_report.md

## 작업
- 비즈니스 문제 정의 (수치 포함)
- MECE 원인 계층 구조 분석
- 핵심 분석 질문 Top 3 확정

## 아웃풋
03_problem_definition.md
```

문제 정의

EDA 패턴 → "지난 분기 대비 마진이 12% 감소했다" 형태의 구체적 문제로
변환

MECE 원인 계층

원인 → 원인의 원인 → 해결방안을 겹치지 않고 빠짐없이 구조화

분석 질문

데이터로 답할 수 있고, 답이 의사결정으로 이어지는 질문 3개 확정

"무엇이 일어났는가"가 아니라 "왜 일어났는가"를 묻는다

@problem — 분석 질문이 흔들리지 않게 정리하기

원인을 겹치지 않고 빠짐없이 나눠야 — 이후 지표 정의와 검증이 흔들리지 않음

@PROBLEM이 만드는 구조 (예시)

문제: Furniture 마진이 4배 낮다

- 원인 1: 고할인 집중 (40%+ 상품이 72% 차지)
 - 원인의 원인: 시즌 프로모션 의존
 - 해결: 카테고리별 할인 한도 설정
 - 목표 지표: 카테고리 마진율
- 원인 2: 저마진 채널 비중 증가
 - 원인의 원인: 온라인 공격 출점
 - 해결: 채널별 마진 기준 재설정
- 원인 3: 운송비·처리비용 상승
 - 해결: 물류 파트너 재협상
 - 목표 지표: 카테고리별 운송비율

왜 MECE 구조인가

겹치지 않음 (Mutually Exclusive)

같은 원인을 두 군데서 분석하지 않는다. @analysis가 중복 없이 정확하게 움직인다

빠짐없음 (Collectively Exhaustive)

놓치는 원인이 없다. 대시보드 액션 아이템이 원인 전체를 커버한다

What → Why → How 3단 흐름

@eda(무엇) → @problem(왜) → @metrics/@analysis(얼마나) → @dashboard(보여주기)

@problem은 방향 설정, @metrics는 그 지표를 계산 가능한 정의로 확정 — 역할이 다름

Step 04 — @metrics

분석 질문 → 이름·계산식·단위·제외조건까지 확정

.CLAUDE/AGENTS/METRICS.MD

```
---
name: metrics
description: 분석 질문과 원인 구조를 받아 KPI를
  정의하고, 선행/파생 지표를 포함한
  지표 체계를 확정하는 에이전트.
  파이프라인의 네 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-haiku-4-5
permissionMode: auto
---

# @metrics

## 인풋
- 03_problem_definition.md
- runs/latest/input/ CSV 컬럼

## 지표 유형
결과(Lagging) · 선행(Leading)
진단(Diagnostic) · 파생(Derived)

## 아웃풋
04_kpi_summary.md
```

모호함 제거

"매출" → "세금 제외 순매출
(월 합계, 전월 대비 증감율 포함)"

좋은 지표 기준

- 상대적 · 비교 가능
- 선행 지표 포함 (결과만이면 너무 늦음)
- 원인 분석 가능 (드릴다운 구조)
- 실제 컬럼으로 계산 가능

파생 지표 자동 생성

수익성 비율 = 이익 / 매출

변화율 = (현재 - 이전) / 이전 × 100

집중도 = 상위 N% 기여 비율

03_.md → 04_kpi_summary.md

Step 05 — @analysis

KPI 데이터 검증 + 인사이트 도출 + 액션 추천

.CLAUDE/AGENTS/ANALYSIS.MD

```
---
name: analysis
description: KPI와 분석 질문을 받아 데이터로 검증하고
  전체 흐름 → 세부 원인 확인 순서로 인사이트와
  액션 가능한 결론을 도출하는 에이전트.
  파이프라인의 다섯 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-haiku-4-5
permissionMode: auto
---

# @analysis

## 인풋
- 03_problem_definition.md
- 04_kpi_summary.md
- input/ CSV (필요 시 직접 읽기)

## 인사이트 형식 (3개 이내)
- 발견: 무엇이 보이는가
- 근거: 어떤 수치가 뒷받침
- 왜 중요한가: 비즈니스 임팩트
- 한계: 표본 · 인과관계 경고

## 아웃풋
05_analysis_report.md
```

General → Specific 흐름

- 1 전체 현황 요약
- 2 주요 분해 (차원 · 그룹별)
- 3 원인 탐색
- 4 이상 · 기회 발견

액션 추천 우선순위

- P1 즉시 — 바로 시작 가능
- P2 2~4주 — 준비 · 검증 필요
- P3 모니터링 — 지표 · 기준 지속 관찰

질문 유형	방법론
왜 낮은가?	분해 분석
무엇이 영향?	상관·회귀

CLIP 03를 마치면

EDA에서 발견한 패턴을 문제 정의 → KPI → 분석 보고서로 번역하는 에이전트 3개 완성

@problem · @metrics · @analysis

이번 챕터에서 만드는 것



@dashboard · @report 제작 + /run-pipeline으로 연결

CLIP 04

대시보드·보고서 에이전트 완성

대시보드와 보고서 단계를 만들고
/run-pipeline으로 전체 흐름을 실행합니다.





Step 06 — @dashboard

대시보드 단계만 Sonnet 사용 — 구조 설계와 문장 품질이 중요

.CLAUDE/AGENTS/DASHBOARD.MD

```
----

name: dashboard
description: 분석 인사이트를 받아 스토리텔링
            구조의 인터랙티브 HTML 대시보드를
            제작하는 에이전트. 여섯 번째 단계.
tools: Read, Write, Bash, Glob
model: claude-sonnet-4-6  ← dashboard 단계만
permissionMode: auto
skills:
  - visualize
  - dashboard-design
----

## 인풋
- 05_analysis_report.md
- dashboard-sample.html (템플릿)

## 아웃풋
06_dashboard.html
```

dashboard 단계 — Sonnet

복잡한 인터랙티브 HTML을 한 번에 정확하게 생성해야 해서 Sonnet 사용.
다른 단계는 haiku로 충분

skills: visualize + dashboard-design

디자인 시스템 규칙(색상·레이아웃·타이포)을 스킬로 주입해서 일관된 퀄리티 유지

템플릿 기반

dashboard-sample.html에서 수치만 교체 — 처음부터 HTML을 새로 만들지 않음

대시보드 설계 원칙

요약 KPI → 1차 분석 → 2차 분석 → 원본 데이터 — 계층 구조로 설계하기



색상 3가지 + 차트 선택 원칙

차트마다 기준이 바뀌지 않도록 최소한의 규칙을 둡니다

색상 원칙 — 의미가 있어야 쓴다

#35c995

좋음 · 긍정 · 안전

#ef7d86

나쁨 · 위험

#efb05d

경고 · 보통

#0a0a0a

KPI 수치 — 색상 절대 금지

색으로 판단하게 되면 수치를 못 읽는다

차트 선택 원칙

목적	차트 유형
추세 · 시계열	라인 차트
범주 비교	정렬된 바 차트
비율 · 구성	스택 바 (파이 차트 지양)
분포	히스토그램
두 지표 동시	듀얼 바 · 버블 (이유 있을 때만)

Step 07 + 완성 — @report · /run-pipeline

임원 보고서 압축 + 7단계 전체를 한 번에 실행

STEP 07 — 마지막 에이전트

@report

```
name: report
description: 분석 전체를 임원용 Executive
  Summary로 압축하는 에이전트.
  파이프라인의 일곱 번째 단계.
model: claude-haiku-4-5
tools: Read, Write, Bash, Glob
---
```

분석 전체를 임원이 **5분 안에 읽을 수 있는** 보고서로 압축

- 임원용 요약 보고서
- 핵심 발견 3가지
- 권고 액션 플랜

스킬 — 7단계를 하나로

/run-pipeline

```
# .claude/skills/run-pipeline/SKILL.md

STEP 1 → @data-ingestion  01_.md
STEP 2 → @eda              02_.md
STEP 3 → @problem          03_.md
STEP 4 → @metrics          04_.md
STEP 5 → @analysis         05_.md
STEP 6 → @dashboard        06_.html
STEP 7 → @report           07_.md
```

산출물 파일이 존재하면 skip, 없으면 실행

실패 시 해당 단계에서 중단 + 재실행 가능

3개 스킬 — 이 파이프라인이 실제로 실행되는 도구

/run-pipeline · dashboard-design · visualize

SKILL 01 /run-pipeline

ai-pipeline-kit/.claude/skills/run-pipeline/SKILL.md

```
---

name: run-pipeline
argument-hint: (없음)
tools: Read, Write, Bash, Glob, Agent
---

STEP 1 → @data-ingestion  01_dataset_profile.md
STEP 2 → @eda              02_eda_report.md
STEP 3 → @problem          03_problem_definition.md
STEP 4 → @metrics          04_kpi_summary.md
STEP 5 → @analysis         05_analysis_report.md
STEP 6 → @dashboard        06_dashboard.html
STEP 7 → @report           07_executive_report.md

# 체크포인트: 산출물 파일 있으면 skip
# 실패 시 해당 단계에서 중단
```

SKILL 02 dashboard-design

ai-pipeline-kit/.claude/skills/dashboard-design/SKILL.md

```
---

name: dashboard-design
tools: Read, Glob, Grep, Write, Bash
---

## 핵심 원칙
- KPI 수치: #0a0a0a 검정 — 색상 절대 금지
- 좋음 #35c995 / 나쁨 #ef7d86 / 경고 #efb05d
- 차트: 순수 SVG만 — CDN 금지
- 이모지 완전 금지 — SVG 아이콘으로 대체
```

SKILL 03 visualize

ai-pipeline-kit/.claude/skills/visualize/SKILL.md

```
---

name: visualize
tools: Read, Glob, Grep, Write, Bash
---

## 차트 선택 원칙
- 추세 → line 차트만
- 범주 비교 → 정렬된 bar 우선
- 분포 → scatter / histogram / box
- 복잡한 차트는 해석 가치 > 비용 때만
```


챕터 전체 요약 — 기억할 4단계

STEP 01 폴더 구조 세팅

`.claude/agents/` `runs/latest/input/` `runs/latest/outputs/`
CLAUDE.md에 파이프라인 흐름 + 실행 규칙 선언

STEP 02 수집·탐색 에이전트

`data-ingestion.md` `eda.md`
`model: haiku` · `permissionMode: auto` · `description` 구체적으로

STEP 03 분석 설계 에이전트

`problem.md` `metrics.md` `analysis.md`
세 에이전트 모두 `haiku` · 이전 단계 파일을 인풋으로

STEP 04 시각화·보고서 에이전트

`dashboard.md` `report.md` `/run-pipeline`
`dashboard`만 **Sonnet** · `skills: visualize + dashboard-design`

파일로 소통 · 순서대로 실행 · 실패하면 그 단계만 · `/run-pipeline`으로 하나로

다음 챕터(CH02)에서는 이 구조를 실제 데이터로 직접 실행합니다. 산출물을 하나씩 확인하고 필요한 단계만 다시 호출하는 흐름을 경험합니다.